

Sample Proposal — CostumeHub (strong)

This is a teaching sample. The theater, the volunteers, the interviews, and the spike result are invented for CS 301R. Read it for *shape, voice, and evidence discipline* — not as a project to copy. In your own proposal, every piece of evidence must be real and yours.

A **weak** proposal for the same project is Sample Proposal — CostumeHub (weak). Read them side by side; that contrast is the whole lesson. The section-by-section guide is The Anatomy of a Software Proposal.

Backstage — a costume and props catalog for volunteer-run community theaters

Author: A. Reyes · CS 301R · Written Proposal (CP3)

Executive summary

Volunteer-run community theaters own thousands of costumes and props and track them in spiral notebooks, shoeboxes of photos, and one long-serving volunteer’s memory. The predictable results: items are re-bought because nobody could tell they were already in the building, borrowed pieces never come back, and when the volunteer who “knows the collection” steps down, the collection effectively resets. I interviewed four wardrobe and props volunteers across three Utah County theaters and watched one strike night; all four described the same failure, and two named specific items they had bought twice. **Backstage** is a phone-first catalog for a theater’s costume and props stock: search what you own from the shop floor, see a photo and a bin location, and check items in and out against a show. The semester MVP is the catalog, search, photos, bin locations, and check-out/return for one theater’s costume stock, seeded with real inventory and tested during an actual production. A six-hour spike settled the scariest unknown — whether a volunteer can photograph and file an item in under a minute on the venue’s weak wifi — at a median of 48 seconds, but only with client-side image downscaling, which is now part of the design. Deliberately out of scope: ticketing, budgets, rentals between theaters, and a native mobile app. What remains after the MVP — props, pull lists, measurements, multi-theater sharing — is real work a contributor can pick up without renegotiating the core.

Problem and users

Who. Wardrobe and props volunteers at small, volunteer-run community theaters — the people who own the collection day to day. At the three theaters I visited, that is one to three people per theater, supported by five to ten show-time volunteers who touch the collection for six weeks and then leave. They are not technical users; two of the four keep their records on paper by preference, not by lack of options.

The need. From four interviews (three wardrobe leads, one props head) and one strike-night observation:

1. **“Do we already own this?” cannot be answered from the shop floor.** [E — 4 of 4] Every person I spoke to described walking racks or calling someone at home. Ruth H., wardrobe lead at a 180-seat house, keeps the collection in a spiral notebook plus a shoebox of printed photos; her answer to “do we have a Victorian bustle” is “give me twenty minutes and I’ll go look.”
2. **Nobody knows who has an item out.** [E — 3 of 4] Two theaters described losing pieces to actors who took them home for alterations. One lead estimated “maybe fifteen things a season” walk off and come back late or not at all.
3. **The collection dies with the volunteer who knows it.** [E — 3 of 4, unprompted in 2] The props head I interviewed inherited a storage room with no records after his predecessor moved away; he described the first season as “rediscovering our own stuff.”
4. **Sizes are unknown until a fitting.** [E — 2 of 4] Real, but less acute than the first three, and the two who raised it had workarounds.

Cost of the status quo. Two of the four named a specific item bought twice — in one case a corset at roughly \$120. That is real money at a theater whose entire costume budget for a show is a few hundred dollars, and it is the argument that gets me in the door with a board.

What I have not established. [A] I have not observed how a *new* show-time volunteer behaves with the collection — only how leads behave. That gap is why “a new volunteer can add an item with no verbal instruction” is one of my success criteria rather than an assumption.

Solution overview

A phone-first web catalog for one theater’s collection. A volunteer standing in the costume shop opens it on their phone, searches “bustle,” sees photos of the three they own with the bin each lives in, and walks to the right shelf. When an actor takes a piece home, the volunteer checks it out against that actor and that show; the item shows as out, with a due date, until it comes back.

This addresses the prioritized needs directly: search plus photos answers need 1, check-out/return answers need 2, and the catalog itself — records that outlive the person who made them — answers need 3. Need 4 (sizes) is a field on the item record, populated when someone happens to know, and nothing more than that this semester.

The design constraint that shapes everything is the setting: a concrete-walled costume shop with weak wifi, volunteers using their own phones, and no IT staff anywhere in the building.

Core features

Prioritized. P0 is the MVP; P1 is the roadmap’s first stop; P2 is honest wishlist.

PRI	FEATURE	SERVES
P0	Item record: photo, name, category, size/measurements (optional), condition, bin location	Needs 1, 3, 4
P0	Search and browse by category, from a phone	Need 1
P0	Photo capture and upload from the phone camera	Need 1
P0	Check-out / return against a person and a show, with a due date	Need 2
P0	Two roles — wardrobe lead (edit anything) and volunteer (add items, check out)	Need 3
P1	Per-show pull list — the set of items assigned to a production	Need 2
P1	Bulk-add screen for typing in an existing paper inventory	Adoption
P2	Props as a distinct category with their own fields	Growth
P2	Measurement history per performer	Need 4

Non-goals

Each of these is a thing a reasonable person would expect, and each is deliberately out:

- **No ticketing, box office, or budgeting.** Those are solved by tools the theaters already pay for; competing with them would trade a problem I have evidence for against one I don’t.

- **No inter-theater rental marketplace.** The most-requested “wouldn’t it be cool” in my interviews, and the fastest way to lose the semester — it needs multi-tenant accounts, trust, and shipping logistics before it helps anyone catalog a single costume.
- **No native mobile app.** A responsive web app reaches every volunteer’s phone with no install step, which matters more here than native polish; an install requirement is friction for a six-week volunteer.
- **No barcode or RFID hardware in v1.** Tempting and genuinely useful later, but it adds hardware the theater must buy and I must support. Typed short IDs on bin tags get most of the benefit for zero dollars.
- **No public-facing catalog.** The collection includes borrowed and donated pieces; publishing it invites questions the theater board should answer, not me.

Cutting the rental marketplace is what makes the rest of this proposal credible. It was in my first draft.

Technical approach

CHOICE	WHY
Django + PostgreSQL	I’ve built two Django apps; the admin gives the wardrobe lead a usable data-repair tool on day one for free. Postgres because the data is relational (items, shows, checkouts) and I don’t want to hand-roll joins.
Server-rendered pages + HTMX	The interactions are small and form-shaped. HTMX gets me responsive check-out without a front-end build pipeline I’d have to maintain alone. Risk: new to me — see the learning plan.
Client-side image downscaling before upload	Direct result of the spike below; not optional.
Images on local disk behind a storage interface	Cheapest thing that works, with the swap to object storage already isolated to one module when the collection outgrows a disk.
A \$6/month VPS, one theater per deployment	No IT staff exists at these theaters; a single small box I can hand over, with documented restore steps, beats an architecture nobody can operate.

Learning plan. HTMX is new to me: ~4 hours (official docs + one tutorial), with a fallback to plain form posts that costs a page reload if it doesn’t take. Image handling on phones — EXIF rotation, HEIC from iPhones — is a known trap I have not hit before: ~3 hours, and the spike already surfaced the first half of it.

Spike — run Sep 24, six-hour box. *Question: can a volunteer photograph and file one item in under 60 seconds on a phone, over the theater’s wifi?* Green line: median ≤60 s over ten items on a throttled 3 Mbps connection. Red line: median >90 s, or repeated upload failures. **Result: green, with a condition.** Naive full-resolution upload gave a median of 71 s and **three of ten uploads failed outright** on the throttled connection. Adding client-side downscaling to 1600 px before upload brought the median to **48 s with zero failures**. The spike converted a hope into a design requirement, and it is why “photos are too slow to bother with” is no longer on my risk list.

Risks and mitigation

RISK	CATEGORY	MITIGATION
HEIC photos from iPhones fail to render, silently breaking the core feature for half the volunteers	Technical	Convert on upload; test on a borrowed iPhone in Week 9, not Week 13. Fallback: reject with a clear message rather than store an unviewable file.
I’m new to HTMX and lose days to it	Skill	4-hour learning box up front; documented fallback to plain forms, which costs polish, not function.

RISK	CATEGORY	MITIGATION
The theater's next production strike — Nov 7 — is my only real test window, and it can move	Schedule	Feature-freeze for that test by Nov 2. If the date slips, fall back to a staged test with the previous show's rack and three volunteers, which I've already confirmed Ruth will host.
Volunteers keep using the notebook and the catalog goes stale	Adoption	The bulk-add screen (P1) exists to make the notebook the <i>seed</i> , not the competition; co-design the item form with Ruth before building it; and one success criterion is deliberately about a volunteer other than me entering items.
The theater cannot operate the app after the semester	Maintenance	One VPS, documented restore, a one-page "add a volunteer" doc in the repo, and a board member named as the account owner before launch.
Django or HTMX version churn breaks the build for a future contributor	Dependency	Pinned versions, a lockfile, and CI that runs on every PR — small surface, few dependencies, chosen partly for that reason.

MVP definition

The honest budget: the course runs at about ten hours a week *including* class, reading, writing, and reviews, and the real building window is roughly five weeks (Weeks 9–13). That is what this MVP is sized against.

In: item records with photo, category, size, condition, and bin location; phone-first search and browse; photo capture with client-side downscaling; check-out and return against a person and a show; two roles; deployed on a real VPS; seeded with **60 real items** from one theater's costume stock.

Out for now: pull lists, bulk import, props-specific fields, measurement history, notifications, and anything multi-theater.

Vertical, not broad: one user type (wardrobe volunteer), one workflow (find it, take it, bring it back), working end to end on real data in a real building.

Success criteria

1. **The twenty-minute question drops under a minute.** In a timed trial, three volunteers each answer four "do we own X?" questions using only their phones; median under 60 seconds, with no walk to the racks.
2. **Sixty real items catalogued by someone who is not me.** Entry has to survive contact with a non-technical volunteer, or the catalog is a demo.
3. **One production's check-outs tracked end to end**, with every item's location known at strike.
4. **A new volunteer adds an item correctly with no verbal instruction** — handed a phone and a costume, nothing else.

Criteria 2 and 4 are the ones I expect to fail first, which is why they're written down.

Word to the reader: everything above traces to my Discovery Notes (four interviews, one observation), my Feasibility & Risk worksheet, the spike log in `spikes/2026-09-24-photo-upLoad.md`, and the calendar. Where I'm guessing, I said so.